

```
# Install required libraries (run once)
!pip install torch torchvision pillow matplotlib requests -q

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.models as models
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np
import copy
import requests
from io import BytesIO
```

```
# Use GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

IMAGE_SIZE = 512 if torch.cuda.is_available() else 256 # smaller on CPU

# Image loader: converts PIL Image → tensor
def load_image(image_path_or_url, size=IMAGE_SIZE):
    transform = transforms.Compose([
        transforms.Resize((size, size)),
        transforms.ToTensor(),
    ])
    if image_path_or_url.startswith("http"):
        response = requests.get(image_path_or_url)
        image = Image.open(BytesIO(response.content)).convert("RGB")
    else:
        image = Image.open(image_path_or_url).convert("RGB")

    # Add batch dimension: (1, C, H, W)
    image = transform(image).unsqueeze(0)
    return image.to(device)

# Tensor → PIL Image for display
def tensor_to_image(tensor):
    image = tensor.cpu().clone().squeeze(0)
    image = image.clamp(0, 1)
    return transforms.ToPILImage()(image)

# Show images side by side
def show_images(content_img, style_img, output_img=None):
    imgs = [("Content", content_img), ("Style", style_img)]
    if output_img is not None:
        imgs.append(("Output (Stylized)", output_img))
    fig, axes = plt.subplots(1, len(imgs), figsize=(6 * len(imgs), 5))
    for ax, (title, img) in zip(axes, imgs):
        ax.imshow(tensor_to_image(img) if not isinstance(img, Image.Image) else img)
        ax.set_title(title, fontsize=14)
        ax.axis("off")
    plt.tight_layout()
    plt.show()
```

Using device: cuda

```
# — OPTION A: Use public URLs (no upload needed) —————
# Content: a photo of a dog (from CIFAR-10-like source)
CONTENT_URL = "https://upload.wikimedia.org/wikipedia/commons/thumb/1/18/Dog_Breeds.jpg/1200px-Dog_Breeds.jpg"

# Style: Van Gogh's Starry Night (WikiArt-style)
STYLE_URL = "https://upload.wikimedia.org/wikipedia/commons/thumb/a/aa/Claude_Monet_-_Water_Lilies_-_1906%2C_Ryerson.jpg/1280px-Claude_Monet_-_Water_Lilies_-_1906%2C_Ryerson.jpg"

content_img = load_image(CONTENT_URL)
style_img = load_image(STYLE_URL)

# — OPTION B: Upload your own files to Colab —————
# from google.colab import files
# uploaded = files.upload() # upload content.jpg and style.jpg
# content_img = load_image("content.jpg")
# style_img = load_image("style.jpg")

print(f"Content image shape: {content_img.shape}")
print(f"Style image shape: {style_img.shape}")
show_images(content_img, style_img)
```

Content image shape: torch.Size([1, 3, 512, 512])
Style image shape: torch.Size([1, 3, 512, 512])

Content



Style



```
# Load VGG19 pretrained on ImageNet
vgg = models.vgg19(pretrained=True).features.to(device).eval()

# Freeze all VGG19 weights – we only optimize the output image
for param in vgg.parameters():
    param.requires_grad_(False)

print("VGG19 loaded and weights frozen.")
print(vgg)
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth" to /root/.cache/torch/hub/checkpoints/vgg19-dcbb9e9d.p
100%|██████████| 548M/548M [00:08<00:00, 66.0MB/s]
VGG19 loaded and weights frozen.
```

```
Sequential(
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): ReLU(inplace=True)
  (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (3): ReLU(inplace=True)
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (5): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (6): ReLU(inplace=True)
  (7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (8): ReLU(inplace=True)
  (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (11): ReLU(inplace=True)
  (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (13): ReLU(inplace=True)
  (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (15): ReLU(inplace=True)
  (16): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (17): ReLU(inplace=True)
  (18): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (19): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (20): ReLU(inplace=True)
  (21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (22): ReLU(inplace=True)
  (23): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (24): ReLU(inplace=True)
  (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (26): ReLU(inplace=True)
  (27): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (28): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (29): ReLU(inplace=True)
  (30): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (31): ReLU(inplace=True)
  (32): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (33): ReLU(inplace=True)
  (34): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (35): ReLU(inplace=True)
  (36): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
```

```

# VGG19 layer names we care about
# Content: deep layer to capture high-level structure
CONTENT_LAYER = "conv4_2"

# Style: multiple layers to capture textures at different scales
STYLE_LAYERS = ["conv1_1", "conv2_1", "conv3_1", "conv4_1", "conv5_1"]

# Map readable names → VGG19 Sequential indices
VGG_LAYER_MAP = {
    "conv1_1": "0", "conv1_2": "2",
    "conv2_1": "5", "conv2_2": "7",
    "conv3_1": "10", "conv3_2": "12", "conv3_3": "14", "conv3_4": "16",
    "conv4_1": "19", "conv4_2": "21", "conv4_3": "23", "conv4_4": "25",
    "conv5_1": "28", "conv5_2": "30", "conv5_3": "32", "conv5_4": "34",
}

def get_features(image, model, layers):
    """
    Pass image through VGG19 and collect activations at specified layers.
    Returns dict: { layer_name: feature_tensor }
    """
    features = {}
    x = image
    for name, layer in model._modules.items():
        x = layer(x)
        # Check if this index matches any requested layer
        for readable_name, idx in VGG_LAYER_MAP.items():
            if name == idx and readable_name in layers:
                features[readable_name] = x
    return features

# VGG19 expects ImageNet-normalized input
def normalize_for_vgg(tensor):
    mean = torch.tensor([0.485, 0.456, 0.406]).to(device).view(1, 3, 1, 1)
    std = torch.tensor([0.229, 0.224, 0.225]).to(device).view(1, 3, 1, 1)
    return (tensor - mean) / std

all_layers = STYLE_LAYERS + [CONTENT_LAYER]

# Extract features from content and style images
content_features = get_features(normalize_for_vgg(content_img), vgg, all_layers)
style_features = get_features(normalize_for_vgg(style_img), vgg, all_layers)

print("Feature maps extracted:")
for name, feat in content_features.items():
    print(f" {name}: {feat.shape}")

```

```

Feature maps extracted:
conv1_1: torch.Size([1, 64, 512, 512])
conv2_1: torch.Size([1, 128, 256, 256])
conv3_1: torch.Size([1, 256, 128, 128])
conv4_1: torch.Size([1, 512, 64, 64])
conv4_2: torch.Size([1, 512, 64, 64])
conv5_1: torch.Size([1, 512, 32, 32])

```

```

def content_loss(target_features, content_features, layer=CONTENT_LAYER):
    """
    MSE between feature maps of the target image and content image
    at a chosen deep layer.
    """
    return torch.mean((target_features[layer] - content_features[layer]) ** 2)

def gram_matrix(tensor):
    # Compute Gram matrix of a feature map.
    # Shape: (batch, channels, H, W) → (channels, channels)
    _, c, h, w = tensor.shape
    tensor = tensor.view(c, h * w)
    gram = torch.mm(tensor, tensor.t())
    return gram / (c * h * w)

# Style loss per layer with equal weighting
def style_loss(target_features, style_features, layers=STYLE_LAYERS):
    # Weighted combination:
    # alpha * content_loss + beta * style_loss
    # Tune alpha/beta to balance content vs style.

    loss = 0
    for layer in layers:
        target_gram = gram_matrix(target_features[layer])

```

```

        style_gram = gram_matrix(style_features[layer])
        loss += torch.mean((target_gram - style_gram) ** 2)
    return loss / len(layers) # average across layers

def total_loss(c_loss, s_loss, alpha, beta):
    return alpha * c_loss + beta * s_loss

print("Loss functions ready.")

```

Loss functions ready.

```

# --- Hyperparameters ---
ALPHA      = 1      # content weight
BETA       = 1e7    # style weight ← key change, balanced ratio
LEARNING_RATE = 0.01
NUM_STEPS  = 2500
SHOW EVERY  = 100

# Start from content image
target = content_img.clone().requires_grad_(True).to(device)
optimizer = optim.Adam([target], lr=LEARNING_RATE)

print(f"Starting style transfer for {NUM_STEPS} steps...")
print(f"Alpha (content): {ALPHA} | Beta (style): {BETA}\n")

loss_history = []

for step in range(1, NUM_STEPS + 1):
    optimizer.zero_grad()
    target.data.clamp_(0, 1)

    target_features = get_features(normalize_for_vgg(target), vgg, all_layers)

    c_loss = content_loss(target_features, content_features)
    s_loss = style_loss(target_features, style_features)
    t_loss = total_loss(c_loss, s_loss, ALPHA, BETA)

    t_loss.backward()
    optimizer.step()

    loss_history.append(t_loss.item())

    if step % SHOW EVERY == 0 or step == 1:
        # Show raw values before weighting so you can actually read them
        print(f"Step [{step:4d}/{NUM_STEPS}] | Total: {t_loss.item():.2f} "
              f"| Content: {c_loss.item():.4f} "
              f"| Style: {s_loss.item():.6f} "          # more decimal places
              f"| Weighted C: {ALPHA*c_loss.item():.2f} "
              f"| Weighted S: {BETA*s_loss.item():.2f}")

print("\nDone!")

```

Starting style transfer for 2500 steps...

Alpha (content): 1 | Beta (style): 10000000.0

Step [1/2500]	Total: 1265.21	Content: 0.0000	Style: 0.000127	Weighted C: 0.00	Weighted S: 1265.21
Step [100/2500]	Total: 36.02	Content: 8.5942	Style: 0.000003	Weighted C: 8.59	Weighted S: 27.43
Step [200/2500]	Total: 22.62	Content: 7.9509	Style: 0.000001	Weighted C: 7.95	Weighted S: 14.67
Step [300/2500]	Total: 18.37	Content: 7.5430	Style: 0.000001	Weighted C: 7.54	Weighted S: 10.83
Step [400/2500]	Total: 16.36	Content: 7.2969	Style: 0.000001	Weighted C: 7.30	Weighted S: 9.07
Step [500/2500]	Total: 15.19	Content: 6.9854	Style: 0.000001	Weighted C: 6.99	Weighted S: 8.20
Step [600/2500]	Total: 14.33	Content: 6.8960	Style: 0.000001	Weighted C: 6.90	Weighted S: 7.43
Step [700/2500]	Total: 13.79	Content: 6.7948	Style: 0.000001	Weighted C: 6.79	Weighted S: 6.99
Step [800/2500]	Total: 13.24	Content: 6.6462	Style: 0.000001	Weighted C: 6.65	Weighted S: 6.59
Step [900/2500]	Total: 12.98	Content: 6.4025	Style: 0.000001	Weighted C: 6.40	Weighted S: 6.58
Step [1000/2500]	Total: 12.53	Content: 6.4694	Style: 0.000001	Weighted C: 6.47	Weighted S: 6.07
Step [1100/2500]	Total: 12.26	Content: 6.2479	Style: 0.000001	Weighted C: 6.25	Weighted S: 6.01
Step [1200/2500]	Total: 12.04	Content: 6.3442	Style: 0.000001	Weighted C: 6.34	Weighted S: 5.69
Step [1300/2500]	Total: 11.70	Content: 6.1960	Style: 0.000001	Weighted C: 6.20	Weighted S: 5.50
Step [1400/2500]	Total: 11.62	Content: 6.0451	Style: 0.000001	Weighted C: 6.05	Weighted S: 5.57
Step [1500/2500]	Total: 11.57	Content: 6.2537	Style: 0.000001	Weighted C: 6.25	Weighted S: 5.32
Step [1600/2500]	Total: 11.01	Content: 6.0652	Style: 0.000000	Weighted C: 6.07	Weighted S: 4.94
Step [1700/2500]	Total: 10.75	Content: 5.9735	Style: 0.000000	Weighted C: 5.97	Weighted S: 4.77
Step [1800/2500]	Total: 10.57	Content: 5.9277	Style: 0.000000	Weighted C: 5.93	Weighted S: 4.65
Step [1900/2500]	Total: 10.60	Content: 5.9483	Style: 0.000000	Weighted C: 5.95	Weighted S: 4.65
Step [2000/2500]	Total: 10.49	Content: 5.9488	Style: 0.000000	Weighted C: 5.95	Weighted S: 4.54
Step [2100/2500]	Total: 10.32	Content: 5.8172	Style: 0.000000	Weighted C: 5.82	Weighted S: 4.50
Step [2200/2500]	Total: 10.15	Content: 5.8291	Style: 0.000000	Weighted C: 5.83	Weighted S: 4.32
Step [2300/2500]	Total: 9.89	Content: 5.7620	Style: 0.000000	Weighted C: 5.76	Weighted S: 4.13
Step [2400/2500]	Total: 9.79	Content: 5.6810	Style: 0.000000	Weighted C: 5.68	Weighted S: 4.10
Step [2500/2500]	Total: 15.22	Content: 7.1233	Style: 0.000001	Weighted C: 7.12	Weighted S: 8.10

Done!

Final image

```
# Find clamp
target.data.clamp_(0, 1)

# Show all three images
show_images(content_img, style_img, target)

# Save output
output_pil = tensor_to_image(target)
output_pil.save("stylized_output.png")
print("Saved stylized image as stylized_output.png")

# Download from Colab
from google.colab import files
files.download("stylized_output.png")
```

Content



Style



Output (Stylized)



Saved stylized image as stylized_output.png

```
plt.figure(figsize=(8, 4))
plt.plot(loss_history, color="steelblue", linewidth=1.5)
plt.title("Total Loss Over Optimization Steps", fontsize=14)
plt.xlabel("Step")
plt.ylabel("Total Loss")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



